

Final Year Project Report
IOT BASED AIR QUALITY INDEX METER SYSTEM



Submitted by:

Huzaifa Shahbaz
21-CE-009

Aqsa Shah
21-CE-039

Supervisor Name:

MAAM SARA TEHSIN
LECTURER

Dated:

21st Jan 2025

Department of Computer Engineering,
HITEC University, Taxila

Table of Contents

ABSTRACT:	7
CHAPTER#1 INTRODUCTION	8
1.1INTRODUCTION:	8
1.2 OBJECTIVES:	8
1.3 PROBLEM STATEMENT:	9
CHAPTER#2 LITERATURE REVIEW/BACKGROUND	10
2.1 LITERATURE REVIEW/BACKGROUND:	10
CHAPTER#3 PROPOSED WORK/METHODOLOGY	15
3.1 PROPOSED WORK/METHODOLOGY:	15
3.2 METHODOLOGY:	15
CHAPTER#4 RESULTS	21
4.1 RESULTS:	21
4.2 ENVIRONEMT:	21
4.2.1 HARDWARE TOOLS:	21
4.2.2 SOFTWARE/DEVELOPMENT TOOLS:	21
4.2.3 SETUP DETAILS:	22
4.3 DATASETS:	23
4.3.1 MATLAB:	23
4.3.1.1 MQ-GAS SENSORS(MQ-5 AND MQ-135) MATLAB CODE:	23
4.3.1.2 PM SENSORS(PM2.5 AND PM10) MATLAB CODE:	25
4.3.1.3 MICROCONTROLLERS(ESP32,ESP8266 AND ARDUINO UNO) MATLAB CODE:	27
4.3.1.4 AQI TABLE INDEX MATLAB CODE:	31
4.3.2 PYTHON:	33
4.3.2.1 AIR QUALITY INDEX PYTHON CODE:	33
4.3.3 ARDUINO FIRMWARE CODING:	34
4.3.3.1 MQ-GAS SENSORS(MQ-5 AND MQ-135)AND HC-05 BLUETOOTH MODULE ARDUINO FIRMWARE (ARDUINO UNO) CODE:	34
4.3.4 BLYNK APP CODING:	36
4.3.4.1 MQ-GAS SENSORS(MQ-5 AND MQ-135)AND HC-05 BLUETOOTH MODULE CODE:	36
4.4 EXPERIMENTATION:	37
4.4.1 MATLAB:	37
4.4.1.1 MQ-GAS SENSORS(MQ-5 AND MQ-135) MATLAB RESULT:	38

4.4.1.2 PM SENSORS(PM2.5 AND PM10) MATLAB RESULT:	38
4.4.1.3 MICROCONTROLLERS(ESP32,ESP8266 AND ARDUINO UNO) MATLAB RESULTS:	39
4.4.1.4 AQI TABLE INDEX MATLAB RESULT:	39
4.4.2 PYTHON:	40
4.4.2.1 AIR QUALITY INDEX PYTHON RESULT:	40
4.4.3 ARDUINO FIRMWARE CODING:	40
4.4.3.1 MQ-GAS SENSORS(MQ-5 AND MQ-135)AND HC-05 BLUETOOTH MODULE ARDUINO FIRMWARE (ARDUINO UNO) CODE:	41
4.4.4 BLYNK APP CODING:	42
4.4.4.1 MQ-GAS SENSORS(MQ-5 AND MQ-135)AND HC-05 BLUETOOTH MODULE CODE:	42
CHAPTER#5 CONCLUSION	43
5.1 CONCLUSION:	43
References	44

List of figures

Figure 3.2.1:ARCHITECTURAL DIAGRAM OF ESP32.....	16
Figure 3.2.2:ARCHITECTURAL DIAGRAM OF ESP8266	16
Figure 3.2.3:ARCHITECTURAL DIAGRAM OF ARUINO UNO	17
Figure 3.2.4:ARITECTURAL DIAGRAM OF MQ-GAS SENSORS(MQ-5 AND MQ-135)	17
Figure 3.2.5:ARCHITECTURAL DIAGRAM OF PM2.5 AND PM10M SENSORS	18
Figure 3.2.6:ARCHITECTURAL DIAGRAM OF WEATHERPROOF HOUSING.....	18
Figure 3.2.7:ARCHITECTURAL DIAGRAM OF ANDROID STUDIO AND FIREBASE	19
Figure 3.2.8:ARCHITECTURAL DIAGRAM OF BLYNK APP	19
Figure 3.2.9:GANTT CHART OF IOT-BASED AIR QUALITY INDEX METER SYSTEM.....	20
Figure 4.4.10:MATLAB RESULTS OF MQ-GAS SENSORS(MQ-135 AND MQ-5)	38
Figure 4.4.11:MATLAB RESULTS OF PM SENSORS(PM2.5 AND PM10)	38
Figure 4.4.12:MATLAB RESULTS OF MICROCONTROLLERS(ESP32,ESP8266 AND ARDUINO UNO)	39
Figure 4.4.13:MATLAB RESULTS OF AQI TABLE INDEX.....	39
Figure 4.4.14:PYTHON RESULTS OF WHOLE WORLD AIR QUALITY INDEX	40
Figure 4.4.15:MQ-GAS SENSOR(MQ-5 AND MQ-135) OF ARDUINO UNO.....	41
Figure 4.4.16:HC-05 BLUETOOTH MODULE (ARDUINO) RESULTS OF ARDUINO UNO	41
Figure 4.4.17:BLYNK APP RESULTS OF MQ-GAS SENSORS(MQ-5 AND MQ-135)	42
Figure 4.4.18:BLYNK APP OF AIR QUALITY INDEX	42

List of Table

Table 2.1: LITERATURE REVIEW OF AIR QUALITY INDEX METER SYSTEM	10
Table 2.2: LITERATURE REVIEW COMPARISONS.....	11
Table 2.3 :COMPARSION OF PROPOSED SYSTEM WITH EXISTING SYSTEM.....	14

ABSTRACT:

The abstract of this project aims to address air pollution, a major concern in urbanization and industrial activities, by designing and implementing an IoT-based Air Quality Index (AQI) meter. The system measures pollutants like PM2.5, PM10, NO2, CO and O3, converting these measurements into a comprehensive AQI value. The collected data is transmitted via a wireless network to a cloud-based platform, where it is processed and visualized. Users can access real-time AQI information via a web interface or mobile application, enabling informed environmental decisions. The system includes a microcontroller for sensor interfacing, a communication module for data transmission, and a cloud server for data storage and analysis. The hardware tools of this project working components which is used are ESP32/ESP8266 , ARDUINO,MQ SERIES GAS SENSOR(MQ-135 OR MQ-5) ,PM2.5 AND PM10 SENSORS,BATTERY PACK ,POWER ADAPTER, CHARGER SETUP,WEATHERPROOF HOUSING and WiFi/BLUETOOTH MODULES. The software tools of this project working components which is used are ANDORID STUDIO, FIREBASE ,BLYNK APP,MATLAB ,PYTHON ,ARDUINO FRAMEWORK ,VS CODE FOR FIRMWARE CODING , THINGS SPEAK ,MQTT PROTOCOLS ,AWS IOT ,ADAFRUIT IO ,GITHUB , AUTHENTICATION MECHANISMS,GAFRANA ,EXCEL, HTTP/HTTPS and SENSOR AND WiFi LIBRARIES.

CHAPTER#1 INTRODUCTION

1.1INTRODUCTION:

In an era, the integration of IOT environmental monitoring systems, particularly Air quality Index(AQI) systems, is crucial for public health and environmental protection. With Urbanization and industrial activities increasing, real-time, accurate air quality data is essential. IOT based AQI systems enable continuous monitoring of air pollutants, providing timely insights and prompt responses to deteriorating air conditions. IOT technology connects sensors to measure air pollutants like PM2.5, PM10, O₃, CO and NO₂. Data is transferred to a central server for analysis and accessibility. This real-time monitoring helps mitigate poor air quality impacts, shapes policies and raises awareness about maintaining a healthy environment. It also plays a crucial role in shaping policies and raising awareness.

IOT technology connects sensors to measure air pollutants like PM2.5, PM10, CO, O₃ and NO₂. Data is transferred to a central server for analysis and accessibility. This real-time monitoring helps mitigate poor air quality impacts, shapes policies and raises awareness about maintaining a healthy environment. It also plays a crucial role in shaping policies and raising awareness.

1.2 OBJECTIVES:

- 1: The study aims to analyze the effects of various air pollutants on public health and the environment, identifying key parameters for AQI measurement and monitoring.
- 2: The study aims to identify suitable sensors and technologies for accurately and reliably detecting pollutants like particulate matter, nitrogen dioxide, and ozone.
- 3: The project aims to create a robust IoT system that uses sensors, microcontrollers, and communication modules to measure and transmit real-time air quality data.
- 4: The study aims to enhance the accuracy, reliability, and efficiency of the AQI meter by comparing the performance of various sensor models and communication protocols.

5: The data is analyzed to generate AQI values, which are presented through a user-friendly web or mobile application, enabling real-time monitoring and alerts for users.

1.3 PROBLEM STATEMENT:

1: The primary issue with this project is its lengthy detection range, which prevents it from accurately displaying the city's air quality reading on the sensors.

2: Urbanization and industrialization have led to a decline in air quality, posing health risks and environmental issues.

3: A project aims to develop an IOT-based AQI system to monitor air pollutants in real-time enabling timely interventions and informed decision-making.

4: The project aims to identify key pollutants, analyze their impact on health and the environment and develop a robust system for continuous monitoring.

5: The solution's business scope includes urban planning, public health management and industrial compliance with primary end-users being environmental agencies, local governments, industries and the general public.

CHAPTER#2 LITERATURE REVIEW/BACKGROUND

2.1 LITERATURE REVIEW/BACKGROUND:

IOT-based Air Quality Index(AQI) systems are gaining popularity due to concerns over air pollution and its impact on public health and the environment. These systems use low-cost, portable sensors to provide real-time on pollutants like PM2.5 and PM10, MQ-135(SO₂,NO₂,O₃ and CO) and MQ-5 (SnO₂,H₂, LPG, CH₄, CO and Alcohol).These sensors are connected via IOT to a central server, which processes and analyzes the data. This approach enhances spatial and temporal resolution of air quality data and facilitates proactive environmental management by providing real-time alerts and insights. The integration of IOT in air quality data index(AQI) systems has the potential to revolutionize environmental monitoring by democratizing access to air quality data. Predictive data analytics powered by machine learning and cloud computing can improve system effectiveness, enabling early warning systems, informed decision making and public awareness. IOT-based AQI systems play a critical role in urban planning, public health management and industrial compliance, necessitating innovative solutions to combat air pollution.

Table 2.1: LITERATURE REVIEW OF AIR QUALITY INDEX METER SYSTEM

STUDY	AUTHORS	YEAR	KEY FINDINGS
Deployment of Low-Cost Sensors	Kumar et al.	2019	The study showcased the efficacy of low-cost, IoT-based sensors in monitoring scalable air quality.
Predictive Analytics in AQI Systems	Smith and Lee	2020	The integration of machine learning for predictive analytics has been highlighted as a method to improve early warning systems.
Real-Time Data for Public Health	Johnson et al.	2021	The use of real-time, IoT-based air quality data has significantly enhanced public health responses.
Enhancing Environmental Awareness	Chen and Patel	2022	The article highlighted the significant role of IoT in enhancing public engagement and raising awareness about air quality issues.

Table 2.2: LITERATURE REVIEW COMPARISONS

Ref #	Published year	Methodology	Results/ Findings	Parameters	Shortcomings/ Gaps
1.	2020	IoT and ML-based smart system for garbage monitoring	Real-time AQI monitoring and fire detection are being implemented for dump yards and garbage management.	Temperature, moisture, light intensity	The application is restricted to garbage management and does not cover general air quality monitoring.
2.	2020	IoT-based environmental quality control system	The method is effective in monitoring environmental pollutants.	Various air pollutants like CO ₂ , SO ₂ .	Large-scale deployments often face high costs and complexity.
3.	2020	IoT framework for urban air quality control	The text focuses on efficient air quality management with solutions that are nearly zero.	O ₃ , NO _x , CO ₂ .	Limited scalability; issues with sensor calibration.
4.	2020	IoT-based indoor air quality control system	The system allows for real-time monitoring and control of indoor air quality.	CO ₂ , VOCs	Indoor only; does not address outdoor air quality monitoring

Proposed System	2021	IoT-based system with Arduino and various sensors	The system monitors various pollutants like O ₃ , SO ₂ , CO, and particulate matter, providing data through a cloud-based interface.	O ₃ , SO ₂ , CO, particulate matter.	The deployment is limited and requires further testing for large-scale outdoor environments and long-term data accuracy.
1.	2018	IoT-based pollution monitoring	The system is continuously monitoring pollution levels through the use of multiple sensors.	CO, particulate matter, smoke.	Limited to predefined areas; lacks user interaction via mobile apps
2.	2018	Edge computing-based IoT architecture	The proposed solution aims to reduce cloud dependency by utilizing low-cost monitoring methods and local data processing.	CO, NO _x , VOCs	The edge computing setup is highly complex and not easily scalable.
3.	2018	IoT-enabled air pollution monitoring and awareness.	Tracks pollutants and raises public awareness through displays and notifications	CO, NH ₃ , particulate matter, smoke	Limited user customization; lacks predictive analytics for future air quality trends
Proposed System	2023	IoT-enabled air pollution meter with smartphone dashboard	The mobile app offers real-time data and alerts on air quality,	CO, NH ₃ , particulate matter, smoke.	The system requires constant internet connection for

			including AQI calculations.		real-time updates and is restricted to areas with smartphone accessibility. The system requires constant internet connection for real-time updates and is restricted to areas with smartphone accessibility.
1.	2020	IoT-based smart environment monitoring	Detects pollutants like CO, smoke, gas; uses LCD for display.	CO, smoke, gas	Data retrieval limited to every 8 hours; no IoT integration.
2.	2008	Environment-al air pollution monitoring with IEEE 1451.	Monitors CO, NO2, SO2, O3 using semiconductor sensors	CO, NO2, SO2, O3	The process necessitates calibration for various gases prior to each measurement
3.	2018	Centralized air pollution detection and monitoring	Uses microcontroller and WiFi module; supports various gas sensing methods	Multiple pollutants (method dependent)	Lacks specific IoT feature integration; manual data evaluation.
Proposed System	2022	Real-time indoor/outdoor monitoring with IoT	Uses ESP32, MQ135, DHT22; displays data via BLYNK app	CO2, temperature, humidity	Limited range due to WiFi module; performance dependent on stable network connection.

Table 2.3 :COMPARSION OF PROPOSED SYSTEM WITH EXISTING SYSTEM

Functionality	Proposed System	Existing System		
	Name	System name 1	System name 2	System name 3
Real-time monitoring	Yes	Yes	No	Yes
Data Accuracy	High	Medium	Low	Medium
Ease of use	Yes	No	Yes	No
Security	Yes	No	Yes	No
Alerts and Notification	Multiple Sensors Supported	Single Sensor	Single Sensor	Multiple Sensors Supported
Historical Data Analysis	Yes	No	No	Yes
Cost Efficiency	Moderate	High	Medium	Low
Scalability	High	Low	Medium	High
Maintenance Requirements	Low	High	Medium	Low

CHAPTER#3 PROPOSED WORK/METHODOLOGY

3.1 PROPOSED WORK/METHODOLOGY:

This is a project that designs and develops an IoT-based Air Quality Index (AQI) Meter System that monitors and analyzes real time critical air pollutant like SO₂,NO₂,O₃,CO SnO₂,H₂, LPG, CH₄, CO and Alcohol. Hardware components of the system comprise sensors(PM2.5 and PM10,MQ-5 and MQ-135), microcontrollers (Arduino Uno, ESP32 and ESP8266), communication module, weatherproof, cost affordable housing, and efficient power supply. Software architecture of the project includes acquisition, processing, storage in the cloud, and visualization, all for user-friendly and accessible interface. Firmware development would utilize VS Code for accurate collection, processing, and transmission of data. The hardware and software will be tested and integrated comprehensively for functional compliance and reliability under various environmental conditions. Such deployment will be more prevalent in urban, industrial, and residential areas, as with cloud-based platforms, real-time data access and visualization will be possible. Maintenance strategies will include sensor calibration with firmware upgrades, as well as an iterative improvement approach to ensure long-term accuracy and reliability. This is expected to yield an efficient, robust, and scalable solution for air quality monitoring in varied environments.

3.2 METHODOLOGY:

In this section, we tell you about the methodology and its architectural diagram of our components and the figure as shown below:

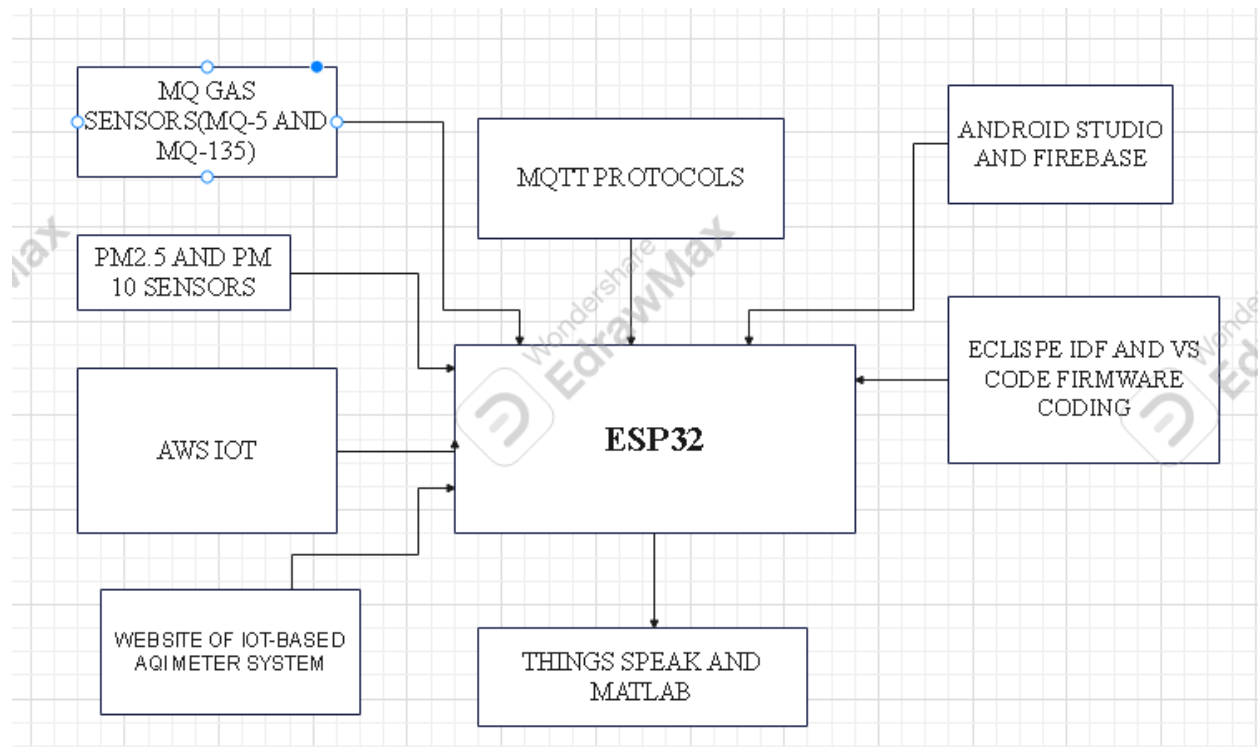


Figure 3.2.1:ARCHITECTURAL DIAGRAM OF ESP32

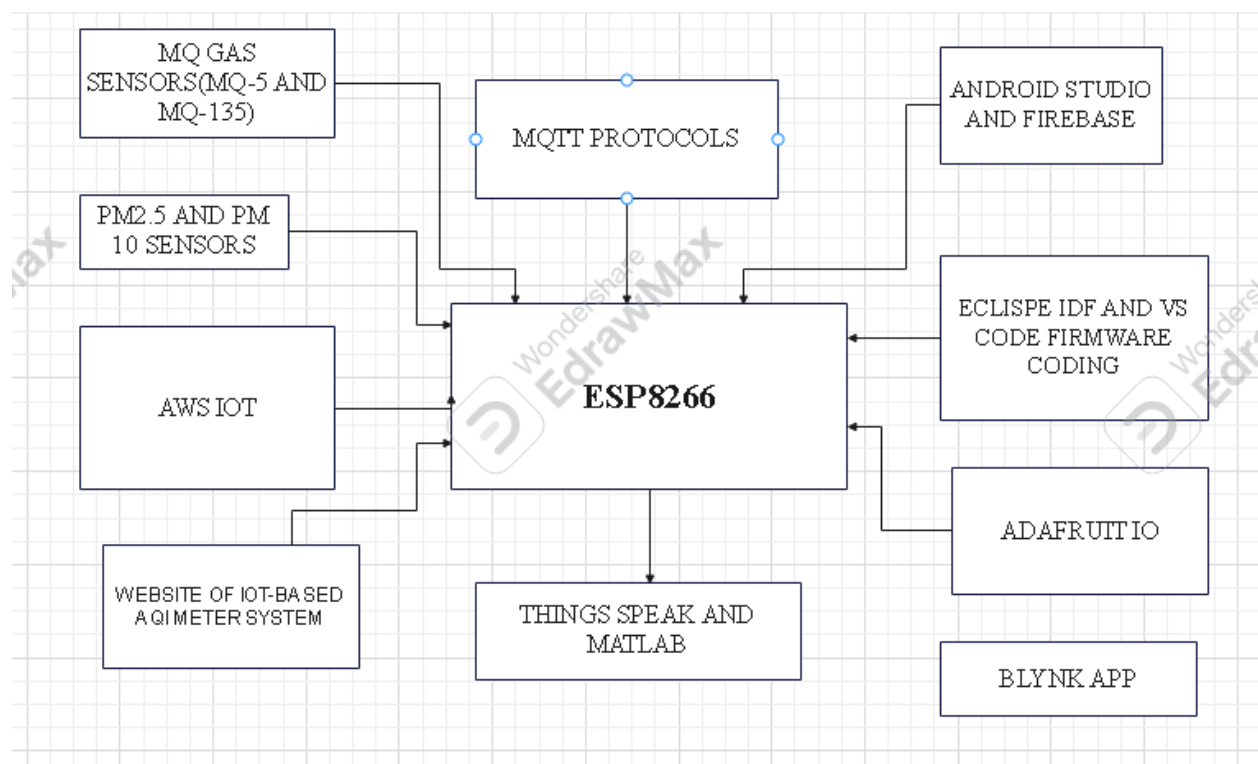


Figure 3.2.2:ARCHITECTURAL DIAGRAM OF ESP8266

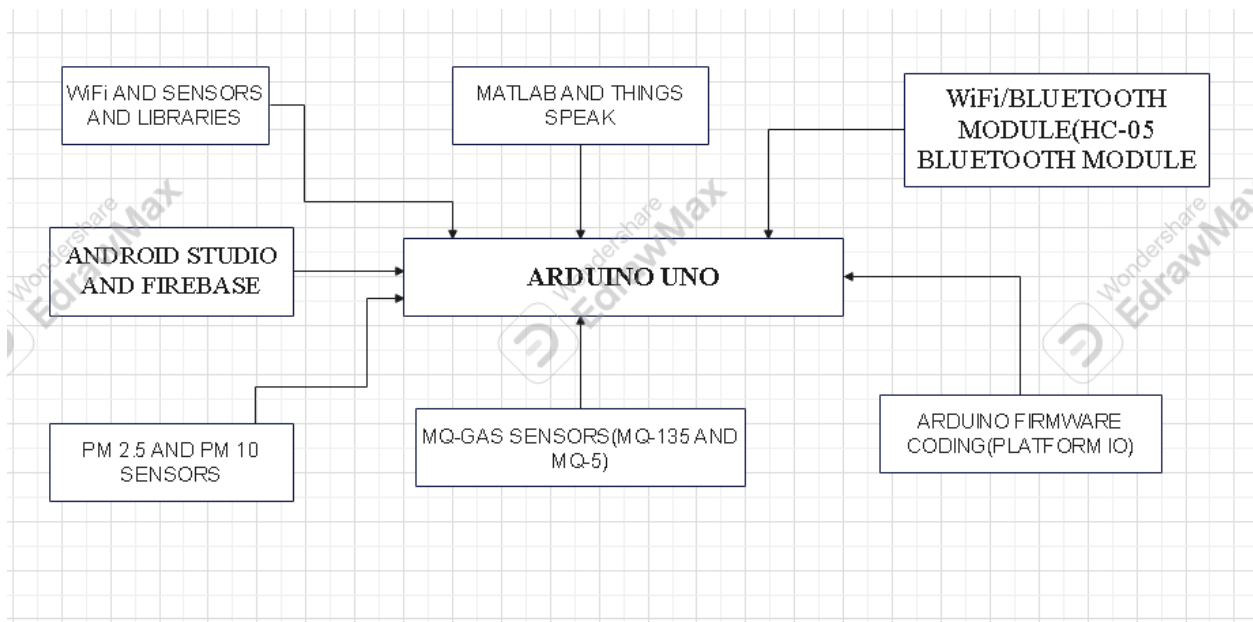


Figure 3.2.3:ARCHITECTURAL DIAGRAM OF ARDUINO UNO

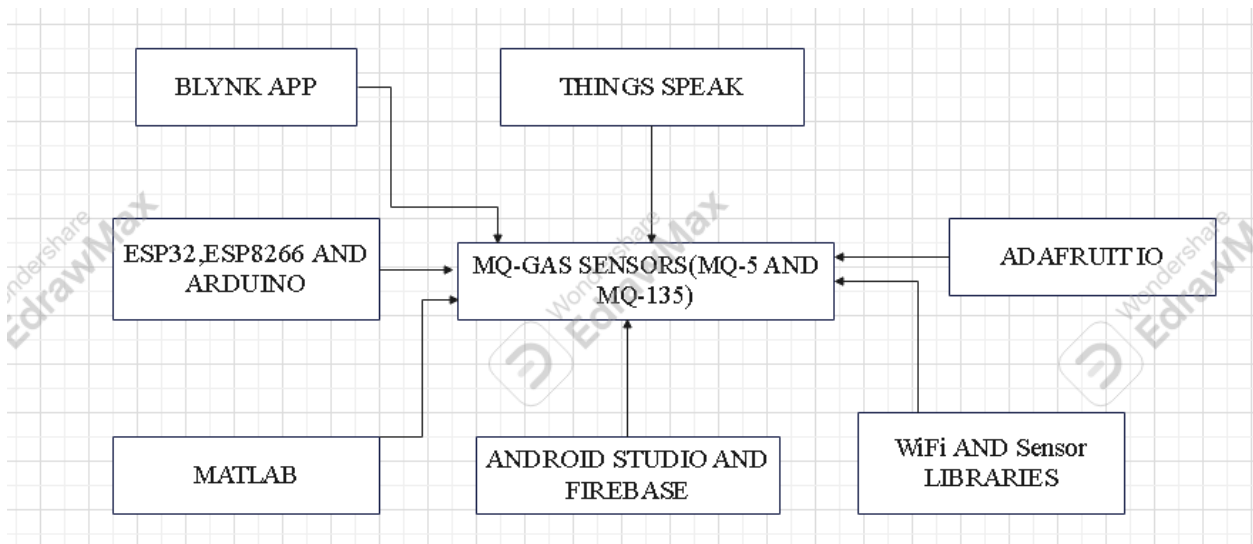


Figure 3.2.4:ARITECTURAL DIAGRAM OF MQ-GAS SENSORS(MQ-5 AND MQ-135)

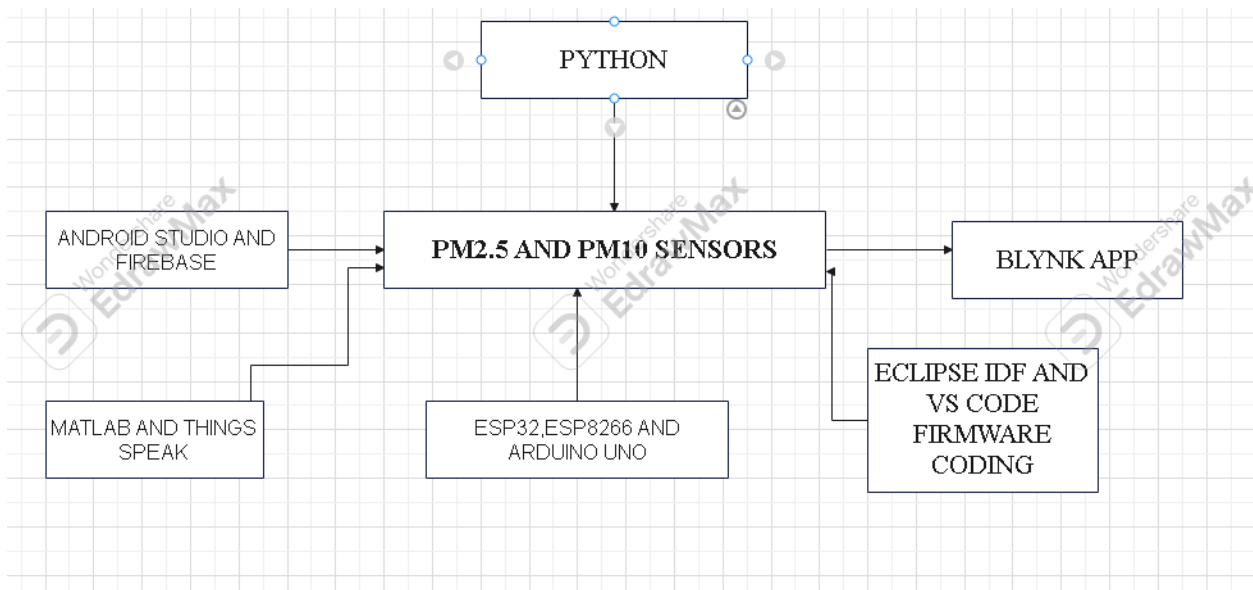


Figure 3.2.5:ARCHITECTURAL DIAGRAM OF PM2.5 AND PM10M SENSORS

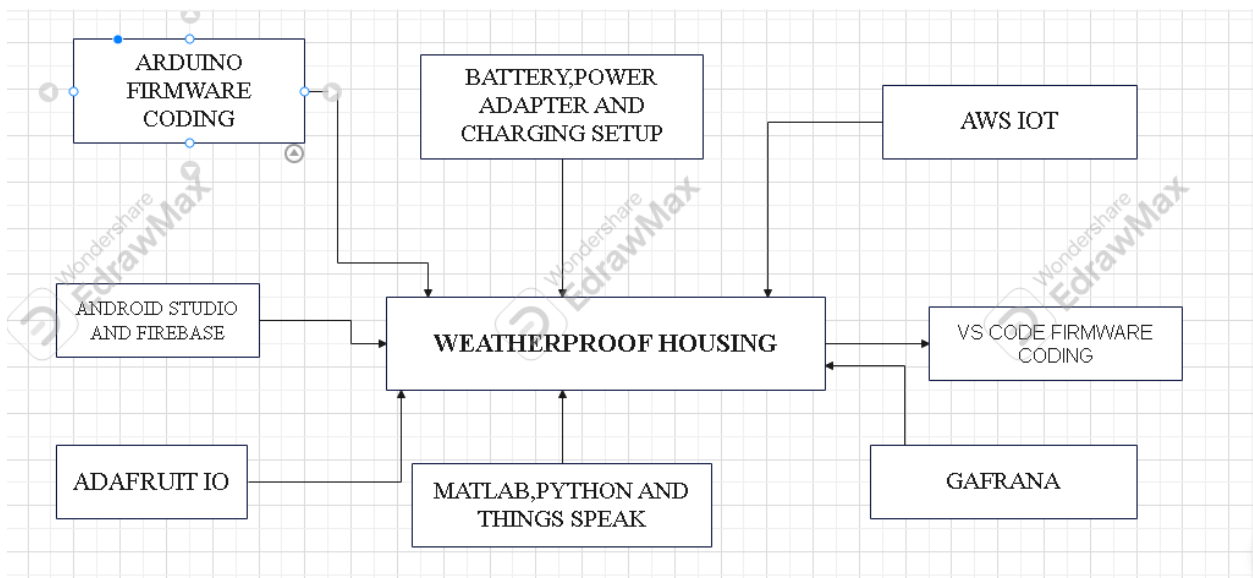


Figure 3.2.6:ARCHITECTURAL DIAGRAM OF WEATHERPROOF HOUSING

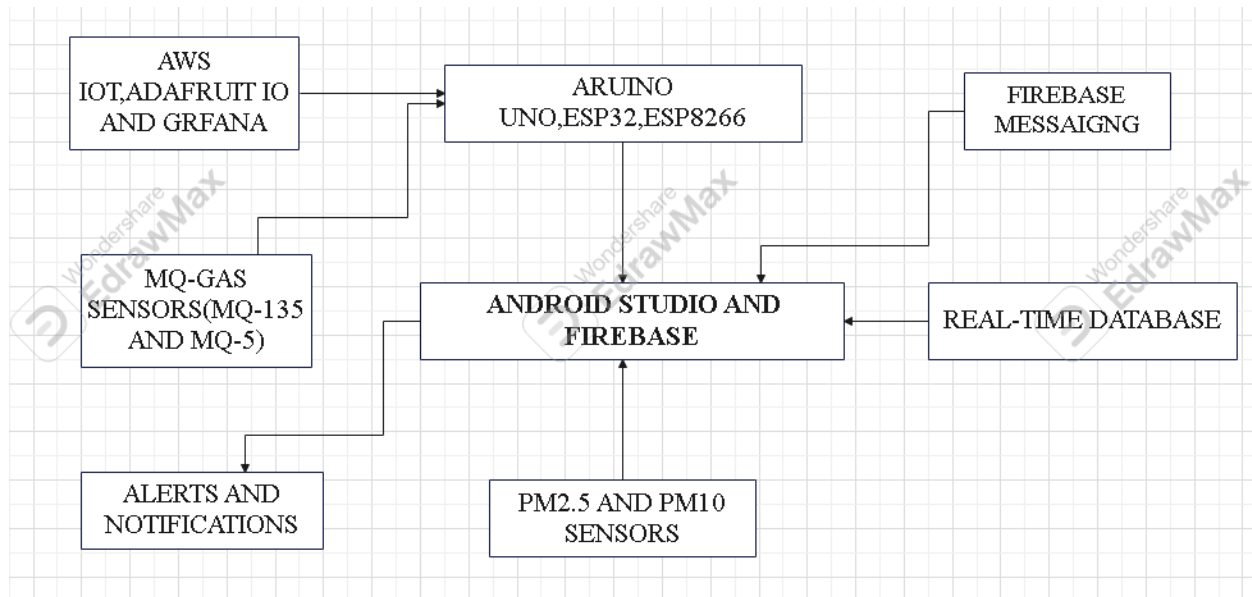


Figure 3.2.7:ARCHITECTURAL DIAGRAM OF ANDROID STUDIO AND FIREBASE

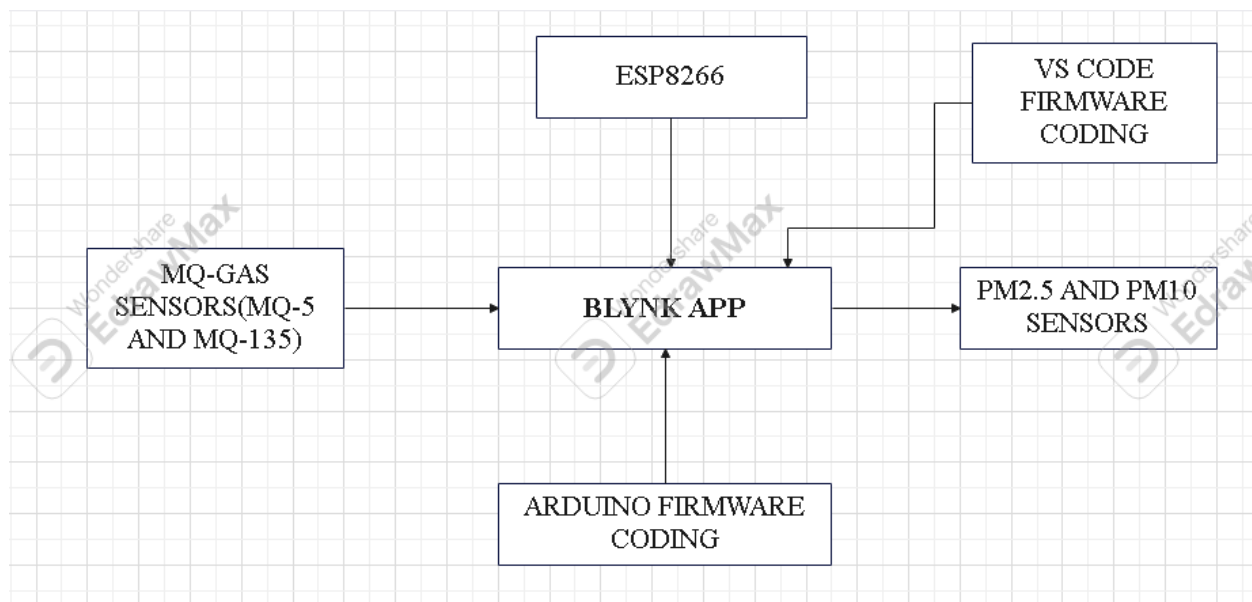


Figure 3.2.8:ARCHITECTURAL DIAGRAM OF BLYNK APP

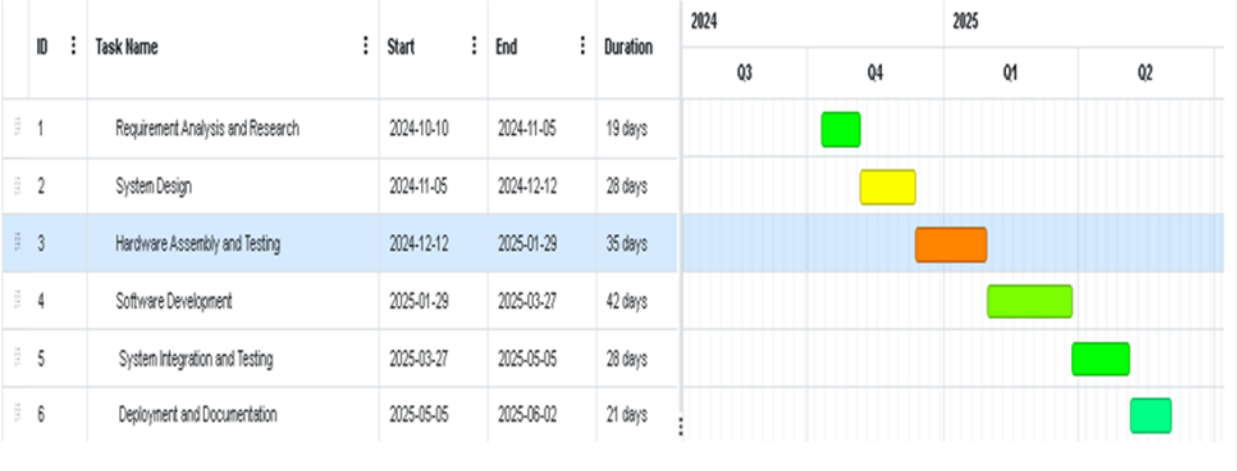


Figure 3.2.9: GANTT CHART OF IOT-BASED AIR QUALITY INDEX METER SYSTEM

CHAPTER#4 RESULTS

4.1 RESULTS:

In this section, we tell you about our final year project results which we had done, and we will talk about our hardware components and software/platforms and code and its result.

4.2 ENVIRONEMT:

In this section, we tell you about the hardware components, software/development and setup details.

4.2.1 HARDWARE TOOLS:

In this section, we tell you about the hardware components of our final year project IOT-BASED AQI METER SYSTEM.

1:ESP32 AND ESP8266

2:ARDUINO UNO

3: 3:MQ SERIES GAS SENSOR(MQ-135 AND MQ-5)

4:PM2.5 AND PM10 SENSORS

5: BATTERY PACK

6: POWER ADAPTER

7: CHARGER SETUP

8: WEATHERPROOF HOUSING (I

9: WiFi/BLEETOOTH MODULES

10:BREADBOARD AND CONNECTING WIRES

4.2.2 SOFTWARE/DEVELOPMENT TOOLS:

In this section, we tell you about the software/development components of our final year project IOT-BASED AQI METER SYSTEM.

1: ANDORID STUDIO AND FIREBASE

2:BLYNK APP

- 3:MATLAB
- 4:PYTHON
- 5: ARDUINO FRAMEWORK AND ECLIPSE IDE
- 6:VS CODE FOR FIRMWARE CODING
- 7: THINGS SPEAK
- 8:MQTT PROTOCOLS
- 9: AWS IOT
- 10:ADAFRUIT IO
- 11:GITHUB
- 12: AUTHENTICATION MECHANISMS
- 13: GAFRANA
- 14:EXCEL
- 15:HTTP/HTTPS
- 16:SENSOR AND WiFi LIBRARIES
- 17:APIs KEY
- 18:PLATFORMIO IDE

4.2.3 SETUP DETAILS:

In this section, we tell you about the setup details of our final year project IOT-BASED AQI METER SYSTEM.

- 1: BATTERY PACK
- 2: POWER ADAPTER
- 3: CHARGER SETUP
- 4: WEATHERPROOF HOUSING
- 5: WiFi/BLUETOOTH MODULES
- 6:BREADBOARD AND CONNECTING WIRES

4.3 DATASETS:

In this section, we will tell you about the codes of our final year project and their working and their results.

4.3.1 MATLAB:

Now we tell you about the matlab coding about their working and the comparison.

4.3.1.1 MQ-GAS SENSORS(MQ-5 AND MQ-135) MATLAB CODE:

```
% MQ-5 and MQ-135 Gas Sensors Data and Plotting
```

```
clc;
```

```
clear;
```

```
close all;
```

```
% Constants (example calibration values from datasheets)
```

```
RL = 10; % Load resistance in kOhms
```

```
Vcc = 5; % Supply voltage in volts
```

```
% MQ-5 Sensor (SnO2, H2, LPG, CH4, CO, Alcohol)
```

```
MQ5_Ro = 10; % Sensor resistance in clean air (kOhms)
```

```
MQ5_Gases = {'SnO2', 'H2', 'LPG', 'CH4', 'CO', 'Alcohol'};
```

```
MQ5_Rs_Ro = [
```

```
    1.2, 1.1, 1.0, 0.9, 0.8, 0.7; % Rs/Ro for SnO2
```

```
    1.1, 0.9, 0.7, 0.6, 0.5, 0.4; % Rs/Ro for H2
```

```
    1.0, 0.8, 0.6, 0.5, 0.4, 0.3; % Rs/Ro for LPG
```

```
    0.9, 0.7, 0.5, 0.4, 0.3, 0.2; % Rs/Ro for CH4
```

```
    0.8, 0.6, 0.4, 0.3, 0.2, 0.1; % Rs/Ro for CO
```

```
    0.7, 0.5, 0.3, 0.2, 0.1, 0.05 % Rs/Ro for Alcohol
```

```

];
MQ5_PPM = [10, 50, 100, 200, 300, 400]; % Concentrations in ppm
% MQ-135 Sensor (SO2, NO2, O3, CO)
MQ135_Ro = 10; % Sensor resistance in clean air (kOhms)
MQ135_Gases = {'SO2', 'NO2', 'O3', 'CO'};
MQ135_Rs_Ro = [
    1.0, 0.9, 0.8, 0.7; % Rs/Ro for SO2
    0.9, 0.8, 0.7, 0.6; % Rs/Ro for NO2
    0.8, 0.7, 0.6, 0.5; % Rs/Ro for O3
    0.7, 0.6, 0.5, 0.4 % Rs/Ro for CO
];
MQ135_PPM = [20, 60, 120, 240]; % Concentrations in ppm
% Calculate Rs and voltage output for MQ-5
MQ5_Rs = MQ5_Ro * MQ5_Rs_Ro; % Sensor resistance for MQ-5
MQ5_Vout = MQ5_Rs ./ (MQ5_Rs + RL) * Vcc; % Voltage output for MQ-5
% Calculate Rs and voltage output for MQ-135
MQ135_Rs = MQ135_Ro * MQ135_Rs_Ro; % Sensor resistance for MQ-135
MQ135_Vout = MQ135_Rs ./ (MQ135_Rs + RL) * Vcc; % Voltage output for
MQ-135
% Plotting MQ-5 Sensor Data
figure;
subplot(2, 1, 1);
hold on;
for i = 1:length(MQ5_Gases)
    plot(MQ5_PPM, MQ5_Vout(i, :), '-o', 'LineWidth', 1.5, 'DisplayName',
MQ5_Gases{i});

```



```

end
title('MQ-5 Sensor (SnO2, H2, LPG, CH4, CO, Alcohol)');
xlabel('Gas Concentration (ppm)');
ylabel('Voltage Output (V)');
grid on;
legend('show');
hold off;

% Plotting MQ-135 Sensor Data
subplot(2, 1, 2);
hold on;
for i = 1:length(MQ135_Gases)
    plot(MQ135_PPM, MQ135_Vout(i, :), '-o', 'LineWidth', 1.5, 'DisplayName',
        MQ135_Gases{i});
end
title('MQ-135 Sensor (SO2, NO2, O3, CO)');
xlabel('Gas Concentration (ppm)');
ylabel('Voltage Output (V)');
grid on;
legend('show');
hold off;

```

4.3.1.2 PM SENSORS(PM2.5 AND PM10) MATLAB CODE:

```

% MATLAB Code for Simulating and Plotting PM2.5 and PM10 Data

% Clear workspace and command window
clear;
clc;

```

```

% Simulate time (in seconds)
simulationTime = 0:1:100; % 100 seconds at 1-second intervals

% Simulate PM2.5 and PM10 sensor data (randomly generated for this example)
% Replace this with real sensor data if available
PM2_5 = 10 + 5*randn(1, length(simulationTime)); % Mean 10, std deviation 5
PM10 = 20 + 8*randn(1, length(simulationTime)); % Mean 20, std deviation 8

% Ensure no negative values in simulated data
PM2_5(PM2_5 < 0) = 0;
PM10(PM10 < 0) = 0;

% Plot the data
figure;
plot(simulationTime, PM2_5, '-o', 'LineWidth', 1.5, 'MarkerSize', 6, 'Color', [0,
0.4470, 0.7410]); % PM2.5 plot

hold on;
plot(simulationTime, PM10, '-s', 'LineWidth', 1.5, 'MarkerSize', 6, 'Color', [0.8500,
0.3250, 0.0980]); % PM10 plot

hold off;

% Add title and labels
title('PM2.5 and PM10 Concentration Over Time', 'FontSize', 14);
xlabel('Time (seconds)', 'FontSize', 12);
ylabel('Concentration (\mug/m^3)', 'FontSize', 12);
legend({'PM2.5', 'PM10'}, 'FontSize', 12, 'Location', 'northwest');

% Add grid for better visualization
grid on;

% Highlight thresholds (if required)
hold on;

```

```

yline(35, '--', 'PM2.5 Threshold', 'Color', [0, 0.4470, 0.7410], 'LineWidth', 1);
yline(50, '--', 'PM10 Threshold', 'Color', [0.8500, 0.3250, 0.0980], 'LineWidth', 1);
hold off;

% Display summary statistics
disp('Summary of PM2.5 and PM10 Data:');
disp(['Mean PM2.5: ', num2str(mean(PM2_5)), ' \mug/m^3']);
disp(['Mean PM10: ', num2str(mean(PM10)), ' \mug/m^3']);
disp(['Max PM2.5: ', num2str(max(PM2_5)), ' \mug/m^3']);
disp(['Max PM10: ', num2str(max(PM10)), ' \mug/m^3']);
disp(['Min PM2.5: ', num2str(min(PM2_5)), ' \mug/m^3']);
disp(['Min PM10: ', num2str(min(PM10)), ' \mug/m^3']);

```

4.3.1.3 MICROCONTROLLERS(ESP32,ESP8266 AND ARDUINO UNO) MATLAB CODE:

```

% Define microcontroller specifications
microcontrollers = {'ESP32', 'ESP8266', 'Arduino UNO'};

% CPU Speed in MHz
cpu_speed = [240, 160, 16];

% RAM in KB
ram = [520, 160, 2];

% Flash Memory in MB
flash = [4, 4, 0.032];

% GPIO Pins
gpio = [36, 17, 14];

% ADC Resolution (bits)
adc = [12, 10, 10];

```

```

% Create figure with subplots
figure('Name', 'Microcontroller Comparison', 'Position', [100, 100, 1200, 800]);

% 1. CPU Speed Comparison
subplot(2,3,1)
bar(cpu_speed)
title('CPU Speed Comparison')
xlabel('Microcontrollers')
ylabel('Speed (MHz)')
set(gca, 'XTick', 1:3, 'XTickLabel', microcontrollers)
grid on

% 2. RAM Comparison
subplot(2,3,2)
bar(ram)
title('RAM Comparison')
xlabel('Microcontrollers')
ylabel('RAM (KB)')
set(gca, 'XTick', 1:3, 'XTickLabel', microcontrollers)
grid on

% 3. Flash Memory Comparison
subplot(2,3,3)
bar(flash)
title('Flash Memory Comparison')
xlabel('Microcontrollers')
ylabel('Flash (MB)')
set(gca, 'XTick', 1:3, 'XTickLabel', microcontrollers)

```

```

grid on
% 4. GPIO Pins Comparison
subplot(2,3,4)
bar(gpio)
title('GPIO Pins Comparison')
xlabel('Microcontrollers')
ylabel('Number of GPIO Pins')
set(gca, 'XTick', 1:3, 'XTickLabel', microcontrollers)
grid on
% 5. ADC Resolution Comparison
subplot(2,3,5)
bar adc)
title('ADC Resolution Comparison')
xlabel('Microcontrollers')
ylabel('Resolution (bits)')
set(gca, 'XTick', 1:3, 'XTickLabel', microcontrollers)
grid on
% 6. Radar Chart for Overall Comparison
subplot(2,3,6)
% Normalize the data for radar chart
normalized_data = [
    cpu_speed./max(cpu_speed);
    ram./max(ram);
    flash./max(flash);
    gpio./max(gpio);

```

```

        adc./max(adc)
];
categories = {'CPU Speed', 'RAM', 'Flash', 'GPIO', 'ADC'};
spider_plot(normalized_data', categories, 'Microcontrollers', microcontrollers);
title('Overall Comparison (Normalized)')
% Add color to all plots
colormap('summer')
% Adjust layout
sgtitle('Microcontroller Specifications Comparison')
set(gcf, 'Color', 'white')
% Function for creating spider plot
function spider_plot(P, categories, legendTitle, legendLabels)
    % Number of variables
    num_vars = size(P, 2);
    % Angles for each axis
    angles = linspace(0, 2*pi, num_vars+1);
    % Plot data points
    colors = lines(size(P, 1));
    % Create the plot
    polarplot([angles(1:end-1), angles(1)], [P(1,:), P(1,1)], '-o', 'LineWidth', 2);
    hold on
    for i = 2:size(P, 1)
        polarplot([angles(1:end-1), angles(1)], [P(i,:), P(i,1)], '-o', 'LineWidth', 2);
    end
    % Fill in the plot

```

```

pax = gca;
pax.ThetaDir = 'clockwise';
pax.ThetaZeroLocation = 'top';
% Set the labels
pax.ThetaTick = rad2deg(angles(1:end-1));
pax.ThetaTickLabel = categories;
% Add legend
legend(legendLabels, 'Location', 'southoutside')
grid on
end

```

4.3.1.4 AQI TABLE INDEX MATLAB CODE:

```

% MATLAB code to define AQI ranges and plot the graph
% Define AQI categories
AQI_Categories = {"Good", "Moderate", "Unhealthy for Sensitive Groups",
"Unhealthy", "Very Unhealthy", "Hazardous"};
AQI_Ranges = [0 50; 51 100; 101 150; 151 200; 201 300; 301 500];
Health_Impacts = {"No threat", "Possibly concerning for sensitive individuals", ...
    "Unhealthy for sensitive groups", "Unhealthy for everyone", ...
    "Entire population likely to be affected", "Serious health effects"};
% Define colors corresponding to each AQI range
Colors = [0 1 0;    % Green
    1 1 0;    % Yellow
    1 0.5 0;    % Orange
    1 0 0;    % Red
    0.5 0 0.5;    % Purple

```

```

        0.3 0 0.3];% Dark Purple
% Create a figure for the bar plot
figure;
hold on;
% Loop through AQI categories and plot bars
for i = 1:length(AQI_Categories)
    bar(i, AQI_Ranges(i, 2), 'FaceColor', Colors(i, :), 'EdgeColor', 'none');
end
% Customize the plot
set(gca, 'XTick', 1:length(AQI_Categories), 'XTickLabel', AQI_Categories);
xlabel('AQI Categories');
ylabel('AQI Range');
title('Air Quality Index (AQI) Categories');
grid on;
ylim([0 550]);
legend(AQI_Categories, 'Location', 'northwest');
% Display AQI table in the command window
disp('AQI Table:');
disp('-----');
for i = 1:length(AQI_Categories)
    fprintf('Category: %-30s | Range: %3d-%3d | Health Impact: %s\n',
        AQI_Categories{i}, AQI_Ranges(i, 1), AQI_Ranges(i, 2), Health_Impacts{i});
end
hold off;

```


4.3.2 PYTHON:

Now we tell you about the python coding about their working and the comparison.

4.3.2.1 AIR QUALITY INDEX PYTHON CODE:

```
import matplotlib.pyplot as plt
import numpy as np
# Define the locations of the cities on the map with placeholder sensor data
cities = {
    "London": (51.5074, 0.1278, 10, 20, {"SO2": 1, "NO2": 5, "O3": 10, "CO": 2}),
    "Paris": (48.8566, 2.3522, 12, 25, {"SO2": 2, "NO2": 3, "O3": 9, "CO": 3}),
    "Tokyo": (35.6895, 139.6917, 8, 18, {"SO2": 1, "NO2": 4, "O3": 7, "CO": 1}),
    "Sydney": (-33.8688, 151.2093, 5, 15, {"SO2": 0, "NO2": 2, "O3": 8, "CO": 0}),
    "Rio de Janeiro": (-22.9068, -43.1729, 15, 30, {"SO2": 3, "NO2": 6, "O3": 12,
"CO": 4}),
    "Moscow": (55.7558, 37.6173, 20, 40, {"SO2": 4, "NO2": 7, "O3": 10, "CO":
6}),
    "Beijing": (39.9042, 116.4074, 35, 60, {"SO2": 5, "NO2": 10, "O3": 15, "CO":
8}),
    "Delhi": (28.6139, 77.2090, 40, 70, {"SO2": 6, "NO2": 12, "O3": 20, "CO":
10}),
    "Cairo": (30.0444, 31.2357, 25, 50, {"SO2": 3, "NO2": 8, "O3": 13, "CO": 7}),
    "Karachi": (24.8607, 67.0011, 30, 55, {"SO2": 4, "NO2": 9, "O3": 16, "CO":
9}),
    "Lahore": (31.5497, 74.3436, 32, 58, {"SO2": 5, "NO2": 10, "O3": 18, "CO":
7}),
    "Islamabad": (33.6844, 73.0479, 20, 40, {"SO2": 2, "NO2": 5, "O3": 10, "CO":
3}),
    "Peshawar": (34.0151, 71.5249, 28, 50, {"SO2": 3, "NO2": 6, "O3": 12, "CO":
5}),
```

```

    "Quetta": (30.1798, 66.9750, 22, 45, {"SO2": 3, "NO2": 7, "O3": 11, "CO": 6}),
}
# Create a scatter plot of the cities with sensor data
plt.figure(figsize=(12, 8))
for city, (lat, lon, pm25, pm10, gases) in cities.items():
    # Scatter the city locations with PM2.5 and PM10 values
    plt.scatter(lon, lat, label=f'{city} (PM2.5: {pm25}, PM10: {pm10})')
    # Optionally, you can print the gas sensor data in the labels
    gas_info = ", ".join([f'{key}: {value}' for key, value in gases.items()])
    print(f'{city}: PM2.5: {pm25}, PM10: {pm10}, Gases: {gas_info}')
# Add labels and title
plt.title("City Locations with Air Quality Sensor Data")
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.legend()
plt.grid()
# Show the plot
plt.show()

```

4.3.3 ARDUINO FIRMWARE CODING:

Now we tell you about the Arduino firmware (Arduino uno) coding about their working and the comparison.

4.3.3.1 MQ-GAS SENSORS(MQ-5 AND MQ-135)AND HC-05 BLUETOOTH MODULE ARDUINO FIRMWARE (ARDUINO UNO) CODE:

```
// Pin definitions
```

```

const int MQ5Pin = A0; // MQ-5 sensor connected to analog pin A0
const int MQ135Pin = A1; // MQ-135 sensor connected to analog pin A1
int MQ5Value = 0;      // Variable to store MQ-5 sensor value
int MQ135Value = 0;    // Variable to store MQ-135 sensor value
// Bluetooth module (HC-05) settings
char incomingByte = 0; // Variable to store incoming Bluetooth data
void setup() {
    Serial.begin(9600);      // Start serial communication at 9600 baud rate
    Serial.println("Sensor readings from MQ-5 and MQ-135 via Bluetooth");
    // Initialize the Bluetooth module (HC-05)
    pinMode(10, OUTPUT);     // RX pin
    pinMode(11, INPUT);      // TX pin
}
void loop() {
    // Read analog values from MQ-5 and MQ-135 sensors
    MQ5Value = analogRead(MQ5Pin);
    MQ135Value = analogRead(MQ135Pin);
    // Send the sensor readings to Bluetooth device via serial communication
    String sensorData = "MQ-5: " + String(MQ5Value) + " | MQ-135: " +
String(MQ135Value);
    Serial.println(sensorData); // Print to serial monitor
    // Send data to Bluetooth device
    if (Serial.available()) {
        incomingByte = Serial.read();
        if (incomingByte == 'A') {

```

```

    Serial.println(sensorData); // Send data to Bluetooth device when 'A' is
received

    }

    }

    delay(2000); // Wait for 2 seconds before taking the next reading

}

```

4.3.4 BLYNK APP CODING:

Now we tell you about the blynk app coding about their working and the comparison.

4.3.4.1 MQ-GAS SENSORS(MQ-5 AND MQ-135)AND HC-05 BLUETOOTH MODULE CODE:

```

#include <Arduino.h>

#include <WiFi.h>          // Use <ESP8266WiFi.h> for ESP8266
#include <BlynkSimpleEsp8266.h> // Use <BlynkSimpleEsp8266.h> for ESP8266
// Blynk credentials

#define BLYNK_TEMPLATE_ID "TMPL6QkR1xiMt"
#define BLYNK_DEVICE_NAME "AIRQULAITINDEX"
#define BLYNK_AUTH_TOKEN
"aCW4BGaSVWaLGuM6qncZy1qQpoXD_6nj"

// Wi-Fi credentials

char ssid[] = "HUZAIFASHAHBAZ";
char pass[] = "huzaf12345786";

// Pin definitions

const int MQ5_PIN = 34;  // GPIO34 for MQ-5
const int MQ135_PIN = 35; // GPIO35 for MQ-135

void setup() {

```

```

Serial.begin(115200);
Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass); // Connect to Blynk Cloud
}
void loop() {
    // Read sensor values
    int mq5_value = analogRead(MQ5_PIN);    // Read MQ-5 value
    int mq135_value = analogRead(MQ135_PIN); // Read MQ-135 value
    // Print to Serial Monitor
    Serial.print("MQ-5 Value: ");
    Serial.println(mq5_value);
    Serial.print("MQ-135 Value: ");
    Serial.println(mq135_value);
    // Send data to Blynk
    Blynk.virtualWrite(V1, mq5_value); // Send MQ-5 data to Virtual Pin V1
    Blynk.virtualWrite(V2, mq135_value); // Send MQ-135 data to Virtual Pin V2
    Blynk.run(); // Keep Blynk connected
}

```

4.4 EXPERIMENTATION:

In this section, we will tell you about the results of our final year project and their working and their results.

4.4.1 MATLAB:

Now we tell you about the matlab results about their working and the comparison.

4.4.1.1 MQ-GAS SENSORS(MQ-5 AND MQ-135) MATLAB RESULT:

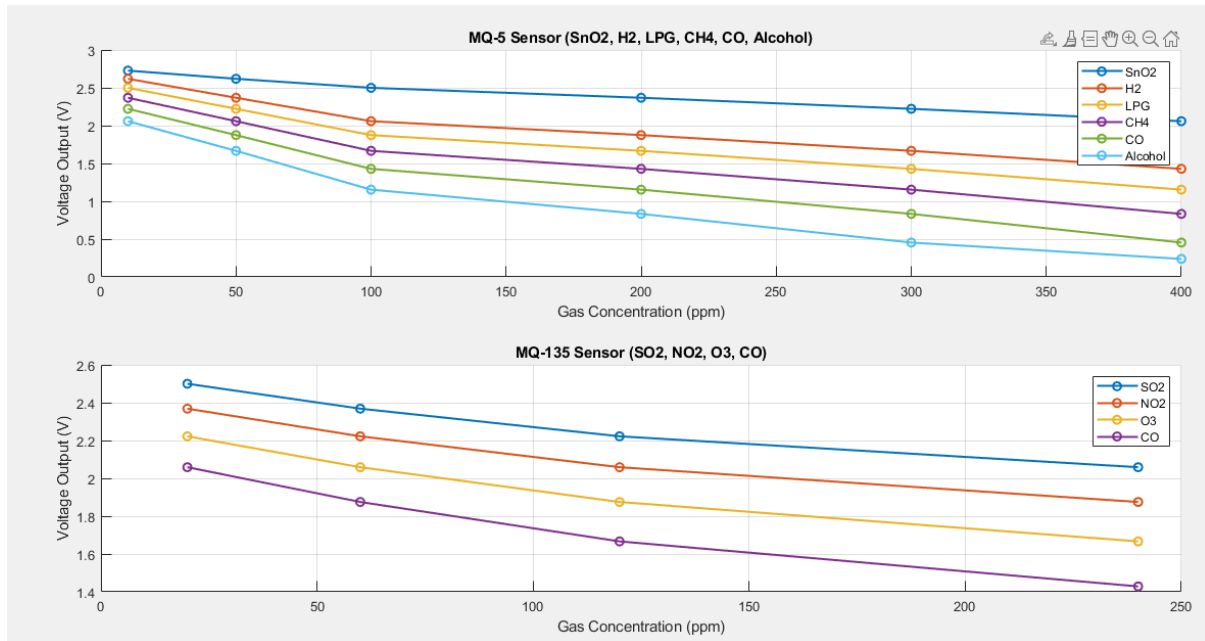


Figure 4.4.10:MATLAB RESULTS OF MQ-GAS SENSORS(MQ-135 AND MQ-5)

4.4.1.2 PM SENSORS(PM2.5 AND PM10) MATLAB RESULT:

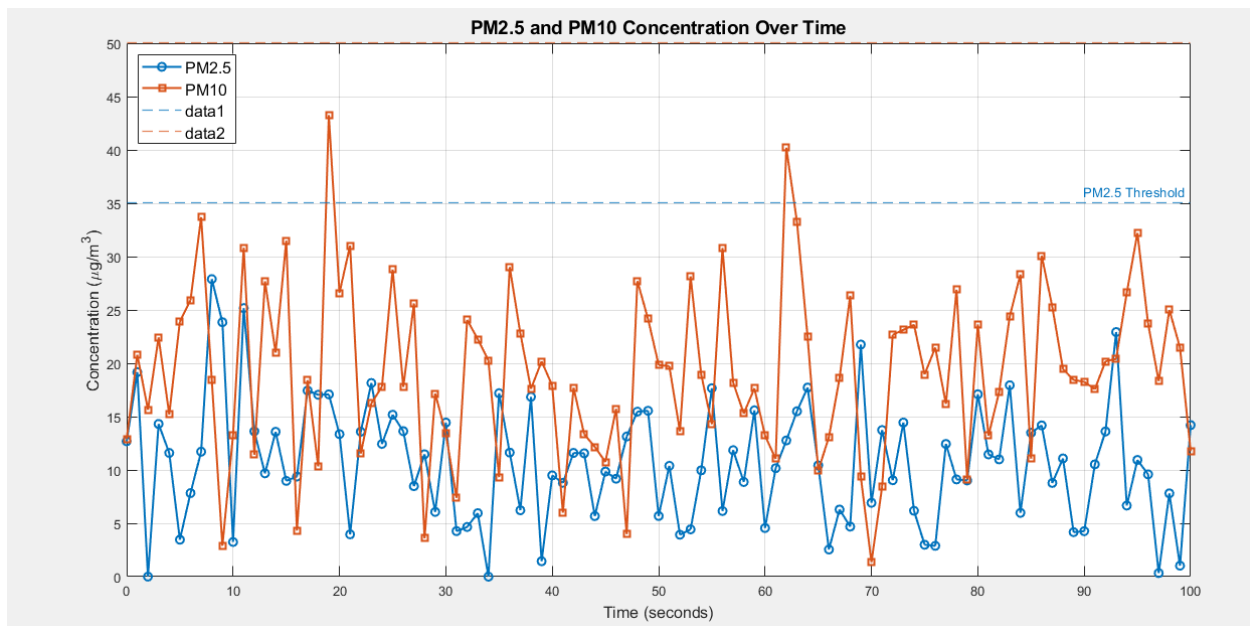


Figure 4.4.11:MATLAB RESULTS OF PM SENSORS(PM2.5 AND PM10)

4.4.1.3 MICROCONTROLLERS(ESP32,ESP8266 AND ARDUINO UNO) MATLAB RESULTS:

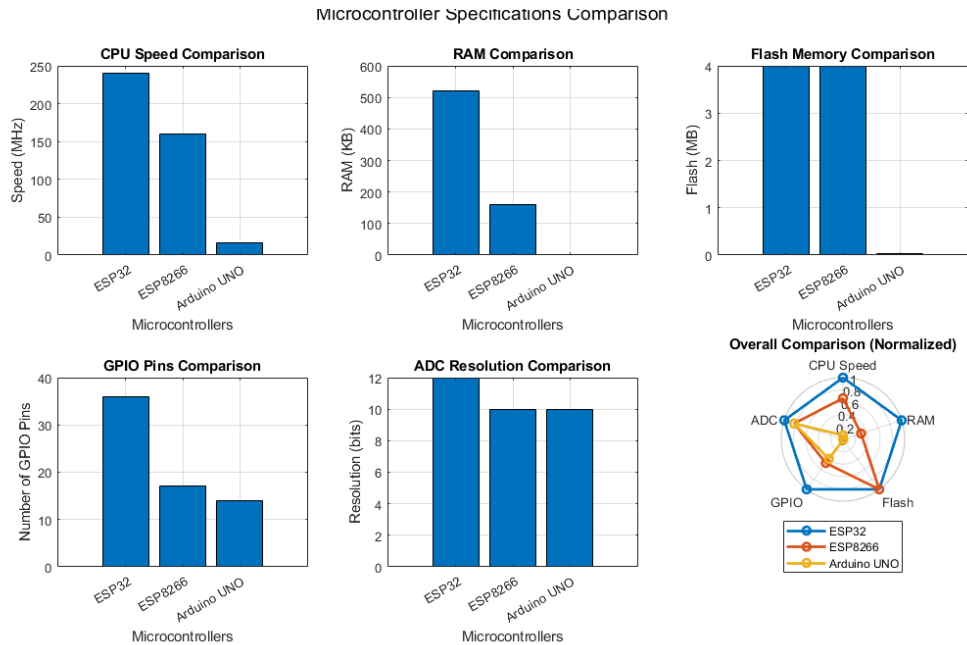


Figure 4.4.12:MATLAB RESULTS OF MICROCONTROLLERS(ESP32,ESP8266 AND ARDUINO UNO)

4.4.1.4 AQI TABLE INDEX MATLAB RESULT:

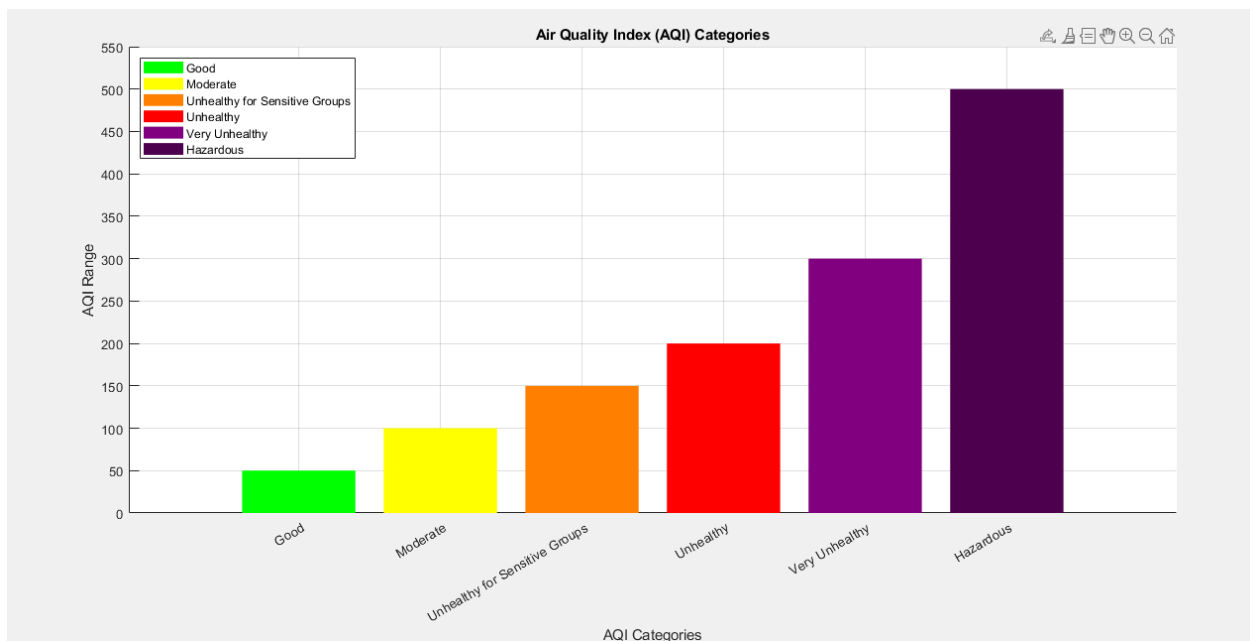


Figure 4.4.13:MATLAB RESULTS OF AQI TABLE INDEX

4.4.2 PYTHON:

Now we tell you about the python results about their working and the comparison.

4.4.2.1 AIR QUALITY INDEX PYTHON RESULT:

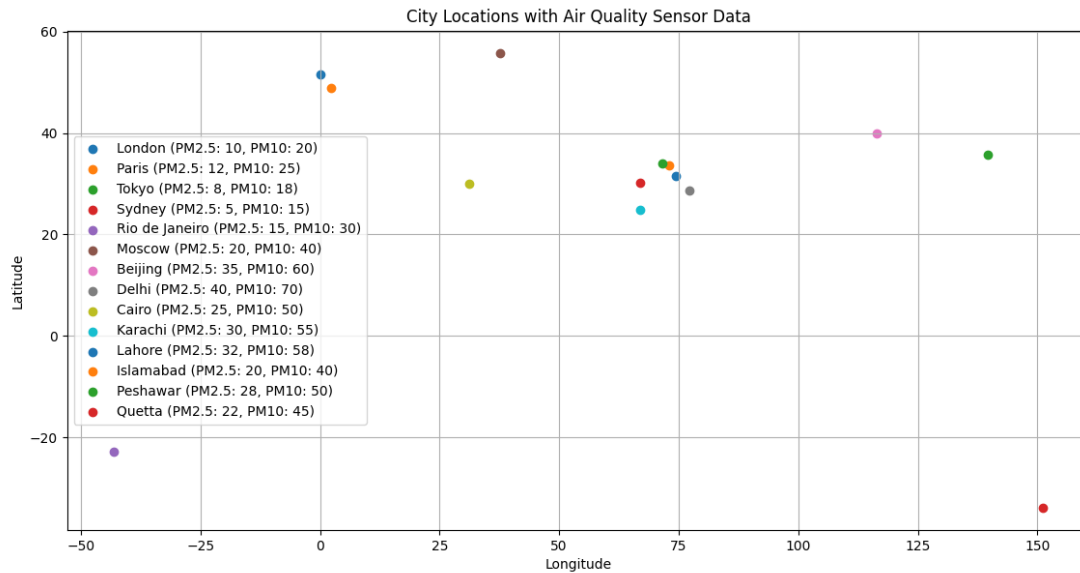


Figure 4.4.14:PYTHON RESULTS OF WHOLE WORLD AIR QUALITY INDEX

4.4.3 ARDUINO FIRMWARE CODING:

Now we tell you about the Arduino firmware (Arduino uno) result about their working.

4.4.3.1 MQ-GAS SENSORS(MQ-5 AND MQ-135)AND HC-05 BLUETOOTH MODULE ARDUINO FIRMWARE (ARDUINO UNO) CODE:

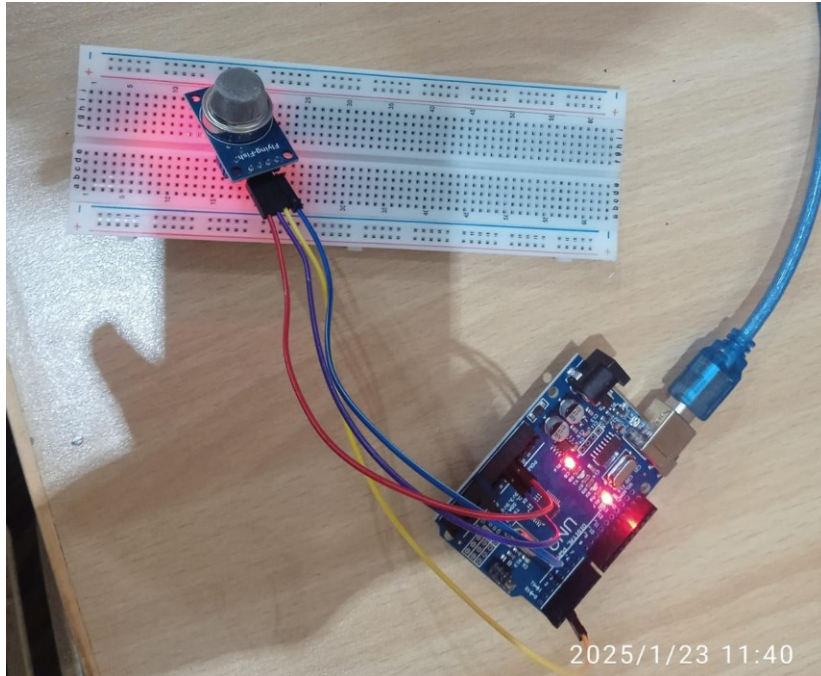


Figure 4.4.15:MQ-GAS SENSOR(MQ-5 AND MQ-135) OF ARDUINO UNO

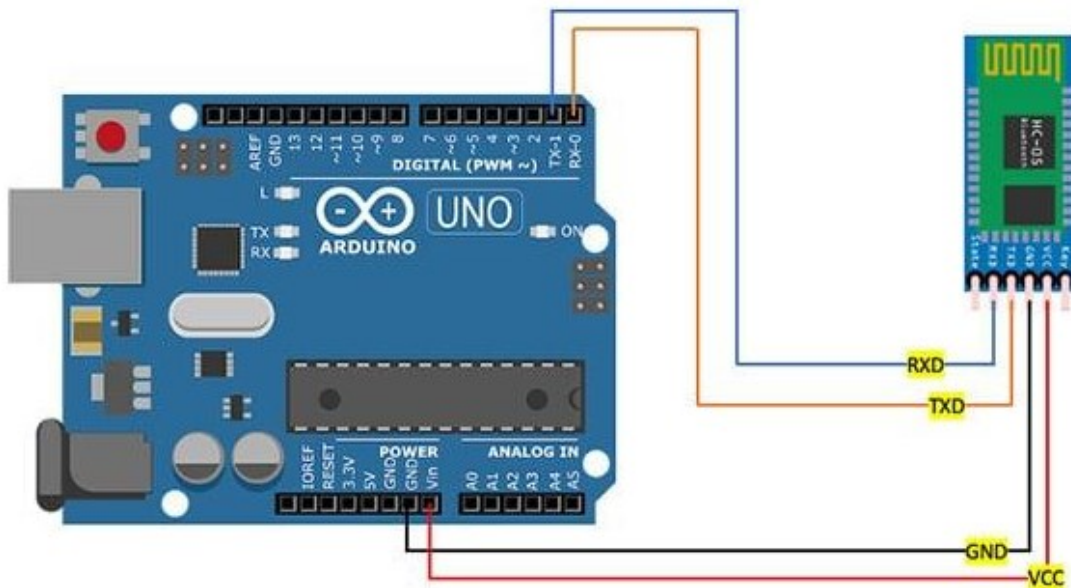


Figure 4.4.16:HC-05 BLUETOOTH MODULE (ARDUINO) RESULTS OF ARDUINO UNO

4.4.4 BLYNK APP CODING:

Now we tell you about the blynk app results about their working and the comparison.

4.4.4.1 MQ-GAS SENSORS(MQ-5 AND MQ-135)AND HC-05 BLUETOOTH MODULE CODE:

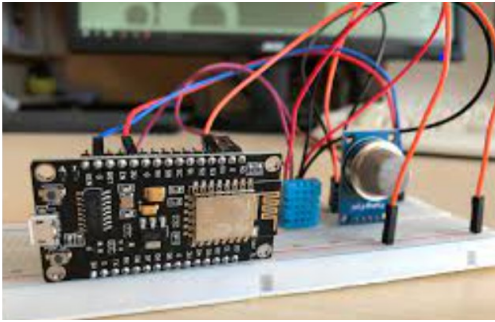


Figure 4.4.17:BLYNK APP RESULTS OF MQ-GAS SENSORS(MQ-5 AND MQ-135)

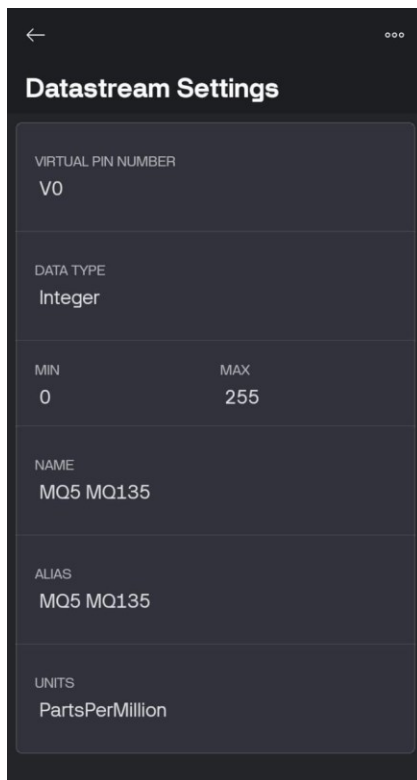


Figure 4.4.18:BLYNK APP OF AIR QUALITY INDEX

CHAPTER#5 CONCLUSION

5.1 CONCLUSION:

In conclusion, we will conclude about our final year project of **IOT-BASED AIR QUALITY INDEX METER SYSTEM**. In this we use are the ESP32, ESP8266, ARDUINO UNO, MQ-GAS SENSORS(MQ-5 AND MQ-135), PM SENSORS(PM2.5 AND PM10), BATTERY PACK, POWER ADAPTER, CHARGER SETUP, WEATHERPROOF HOUSING, WiFi/BLEETOOTH MODULES(HC-05 BLEETOOTH MODULE FOR ARDUINO UNO) and BREADBOARD AND CONNECTING WIRES. With the software tools for our final year project of **IOT-BASED AIR QUALITY INDEX METER SYSTEM**. The software/development tools we use are the ANDROID STUDIO AND FIREBASE, BLYNK APP, MATLAB, PYTHON, ARDUINO FRAMEWORK, VS CODE FOR FIRMWARE CODING, THINGS SPEAK, MQTT PROTOCOLS, AWS IOT, ADAFRUIT IO, GITHUB, AUTHENTICATION MECHANISMS, GAFRANA, EXCEL, HTTP/HTTPS, SENSOR AND WiFi LIBRARIES, APIs KEY and PLATFORMIO IDE. Our project target is to control good health and well-being, sustainable cities and communities, climate actions, industry, innovation and infrastructure, life on land, affordable and clean energy, responsible consumption and production. Our aim is to eliminate the harmful gases in our country as well as the whole world.

References

- 1: Dineshkumar, T., V. Suresh Babu, Pachaivannan Partheeban, and R. Puviarasi. "Air Quality Monitoring System Based on IoT." **Journal of Physics: Conference Series** 1964, no. 6 (2021): 062081.
- 2: Hemalatha, M., R. Arunprasath, J. Shalini Priya, A. M. Shivashankari, K. Rajkumar, and R. Jeyashri. "An IoT Enabled Air Pollution Meter with Digital Dashboard on Smartphone." In *2023 Intelligent Computing and Control for Engineering and Business Systems (ICCEBS)*, IEEE, 2023.
- 3: Yusof, Khairul Huda, M. Abdulrazaq, Fadhilah Aman, M. N. Mohammed, Amirul Asyraf, Ahmad Sukri Ahmad, and Mohamad Syahrul Zahwan Mohd Zabidi. "Design and Development of Real Time Indoor and Outdoor Air Quality Monitoring System Based on IoT Technology." In *2022 IEEE 18th International Colloquium on Signal Processing & Applications (CSPA 2022)*, Selangor, Malaysia, May 12, 2022. IEEE.
- 4: Yusof, K. H., Abdulrazaq, M., Aman, F., Mohammed, M. N., Asyraf, A., Sukri, A., Zabidi, M. S. M., & Alayham, A. R. (2022). Design and Development of Real Time Indoor and Outdoor Air Quality Monitoring System Based on IoT Technology. *2022 IEEE 18th International Colloquium on Signal Processing & Applications (CSPA 2022)*, 12 May 2022, Selangor, Malaysia.
- 5: Sharmila, A. R., J. J. Edwin, T. Sivakumaran, A. P. Prabhu, and R. T. Rajesh. 2022. "Design and Development of Real Time Indoor and Outdoor Air Quality Monitoring System Based on IoT Technology." In **Proceedings of the 2022 IEEE 18th International Colloquium on Signal Processing & Applications (CSPA 2022)**, 187-192. Batu Ferringhi, Malaysia: IEEE.
- 6: IoT Based Air Quality Monitoring System Using Arduino Shazia Afroze 1 , Md. Istiak Hossain Paran 2 , Rakibul Hasan Roki 3 1Assistant Professor, Dept. of Electrical & Electronic Engineering, Stamford University Bangladesh (SUB) 2,3Student, Dept. of Electrical & Electronic Engineering, Stamford University Bangladesh (SUB).
- 7: IoT Based: Air Quality Index and Traffic Volume Correlation [Omar Alruwaili](#); [Ivica Kostanic](#); [Ali Al-Sabbagh](#); [Hamad Almohamedh](#).
<https://ieeexplore.ieee.org/document/9298176/references#references>
- 8: IoT-based_air_quality_monitoring_systems_for_smart [Danny Múnera](#) [Diana P. Tobon V. Johnny Aguirre](#) [Natalia Gaviria](#). https://www.researchgate.net/publication/350823548_IoT-based_air_quality_monitoring_systems_for_smart_cities_A_systematic_mapping_study
- 9: IoT-based_AQI_Estimation_using_Image_Processing_and_Learning_Methods [Nitin Nilesh](#); [Ishan Patwardhan](#); [Jayati Narang](#); [Sachin Chaudhari](#). <https://ieeexplore.ieee.org/document/10152272>
- 10: United Nations. (2015). *Transforming our world: The 2030 Agenda for Sustainable Development*. United Nations. <https://sdgs.un.org/2030agenda>
- 11: World Health Organization. (2018). *Air pollution and health*. WHO. [https://www.who.int/news-room/fact-sheets/detail/ambient-\(outdoor\)-air-quality-and-health](https://www.who.int/news-room/fact-sheets/detail/ambient-(outdoor)-air-quality-and-health)
- 12: United Nations Environment Programme. (2021). *Environmental impact assessment and the SDGs: Supporting the 2030 Agenda*. UNEP. <https://www.unep.org/resources/report/environmental-impact-assessment-and-sdgs-supporting-2030-agenda>

- 13: United Nations Economic and Social Council. (2020). *SDG indicators: Global indicator framework for the 2030 agenda for sustainable development*. United Nations.
<https://unstats.un.org/sdgs/indicators/indicators-list>
- 14: International Energy Agency. (2020). *Energy efficiency 2020*. IEA.
<https://www.iea.org/reports/energy-efficiency-2020>
- 15: Hasan, A., & Ramli, M. (2022). Design and development of real-time indoor and outdoor air quality monitoring system based on IoT technology. *2022 IEEE 18th International Colloquium on Signal Processing & Applications (CSPA 2022)*. <https://doi.org>
- 16: Raj, P., & Evangeline, D. (2019). *Internet of things: Principles and paradigms*. Morgan Kaufmann.
- 17: Espressif Systems. (2021). *ESP32 technical reference manual*. Espressif Systems.
<https://docs.espressif.com>
- 18: Espressif IoT Development Framework (ESP-IDF). <https://docs.espressif.com/projects/esp-idf/en/latest/>
- 19: Air Quality Index Basics. <https://www.airnow.gov/aqi/aqi-basics/>
- 20: Banzi, M., & Shiloh, M. (2022). *Getting started with Arduino: The open source electronics prototyping platform* (3rd ed.). O'Reilly Media.
- 21: Ali, A., & Khan, R. (2022). Design and Development of Real Time Indoor and Outdoor Air Quality Monitoring System Based on IoT Technology. *2022 IEEE 18th International Colloquium on Signal Processing & Applications (CSPA)*.
- 22: Kumar, P., et al. (2020). *IoT-Based Air Pollution Monitoring System*. *International Journal of Engineering Research & Technology (IJERT)*.
- 23: Design and Development of Real-Time Indoor and Outdoor Air Quality Monitoring System Based on IoT Technology" from IEEE CSPA 2022.
- 24: IoT-Based Smart Air Pollution Monitoring System" (IEEE Xplore, 2021).
- 25: **Design and Development of Real-Time Indoor and Outdoor Air Quality Monitoring System Based on IoT Technology**. (2022). IEEE 18th International Colloquium on Signal Processing & Applications (CSPA 2022).
- 26: **Implementation of IoT for Air Quality Monitoring System**. (2020). International Journal of Environmental Science and Technology.
- 27: **IoT-Based Environmental Monitoring Systems: A Review**. (2021). Elsevier Journal of Cleaner Production

